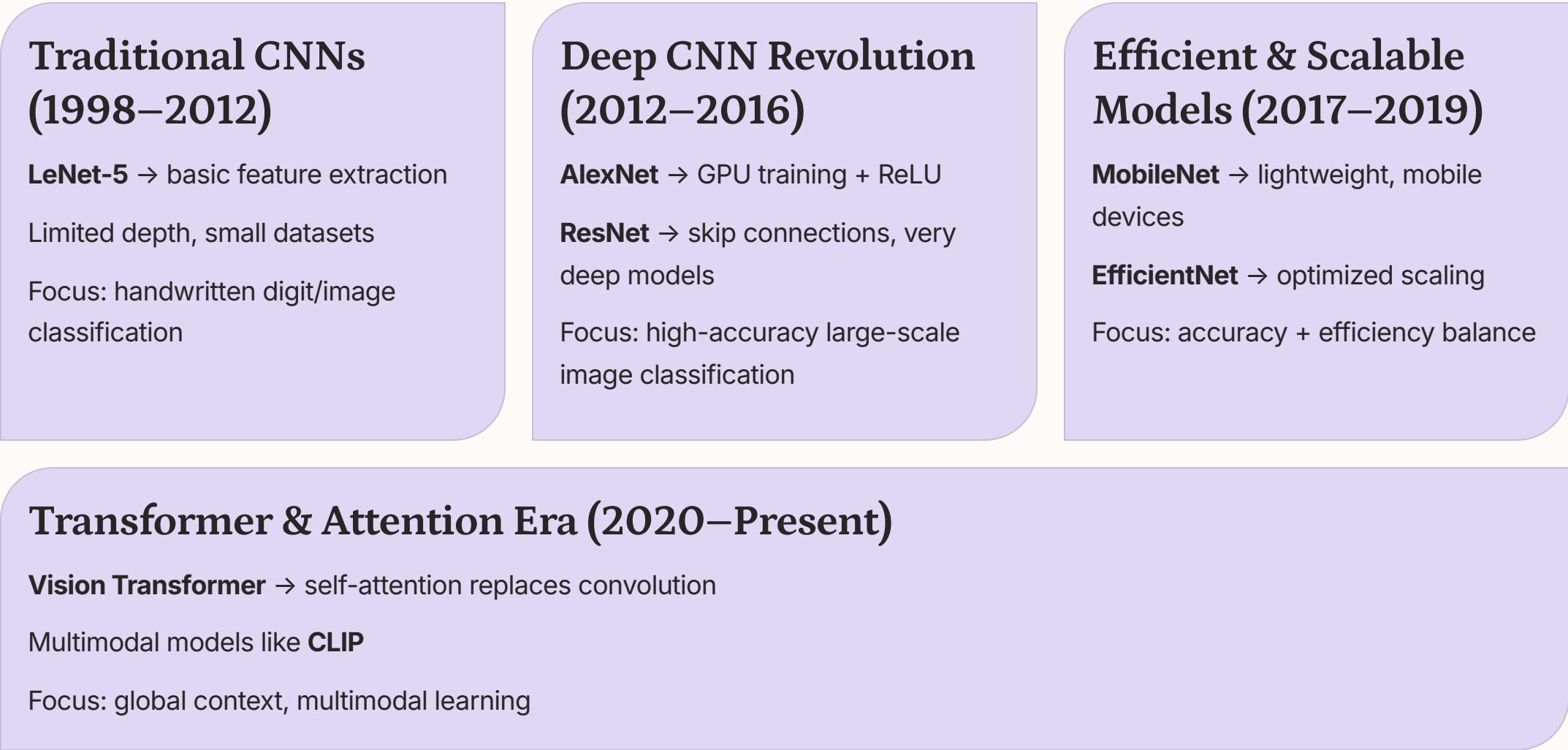




Module III: Convolutional Neural Networks and Computer Vision Architectures

Evolution from traditional convolutional neural networks to deep vision systems



Evolution of Vision Models

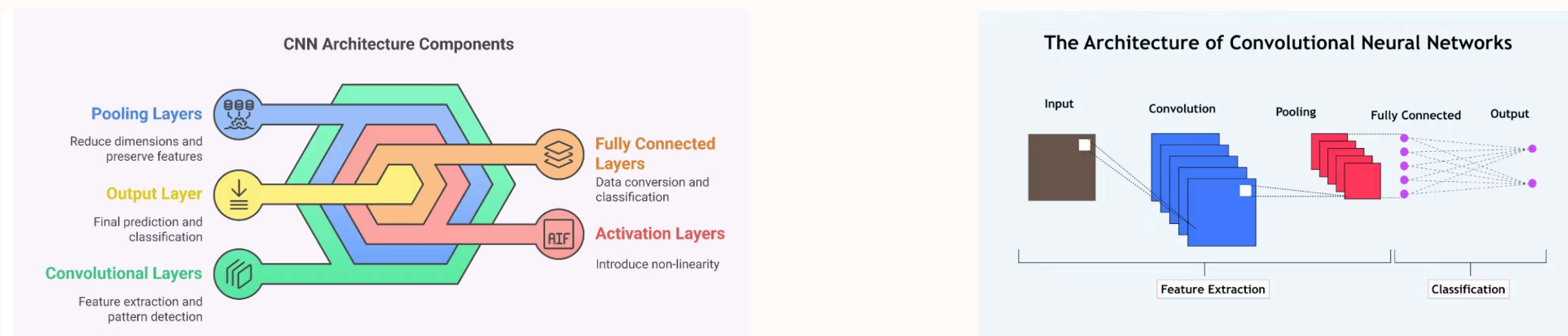
Era	Milestone	Key Innovation
1990s	LeNet-5	Convolution + Subsampling (Pooling)
2012	AlexNet	ReLU + GPU Training + Deep (8 layers)
2014	VGGNet	Very Deep Networks (Stacked 3×3 Convs)
2014	Inception	Multi-scale Convolutions (Parallel Filters)
2015	ResNet	Skip Connections (Up to 152 layers)
2019	EfficientNet	Compound Scaling (Depth + Width + Resolution)
2020s	Vision Transformer / Hybrid Models	Global Attention Mechanism

Architectural design principles of modern CNNs

Core CNN Layers in Architecture

A basic architecture of convolutional neural network works in five clear steps. Each layer in CNN has a simple job. Together, they turn raw images into class scores. These layers of CNN appear in almost every CNN you study, no matter how small or deep.

CNN Architecture Components



Convolution Layer

The convolution layer is the entry point of the simple CNN architecture. Instead of looking at the whole image at once, the model focuses on small blocks of pixels. This helps it notice local patterns before building a full understanding of the image. It uses multiple filters, and each one learns a different visual pattern such as an edge, corner, or texture.

How It Works:

A filter slides across the image one small step at a time. At each step, it multiplies its values with the pixel values under it. The results are added and stored in a new grid called a feature map. This feature map shows where the learned pattern appears in the image. During training, the CNN adjusts the values inside each filter. Over time, the filters become good at detecting useful patterns. Early filters catch simple shapes. Deeper filters catch more complex textures and visual details.

Key actions in this layer

Scanning small regions to understand local structure
Extracting edges, corners, textures, and simple shapes
Producing feature maps that highlight important visual details

Activation Layer

After the convolution layer picks up basic patterns, the model needs a way to understand more complex shapes and relationships. The activation layer makes this possible by adding nonlinearity. Without this step, an architecture of CNN in deep learning would behave like a simple linear model and would miss many details found in real images.

ReLU is the most widely used activation in convolutional neural network architecture. It keeps positive values and removes negative ones. This helps the model focus on strong signals and train faster. Sigmoid and Tanh are also used, mainly when the model needs smoother transitions between values.

How It Works

The activation function is applied to every value in the feature map. If you use ReLU, negative values become zero while positive values stay as they are. This simple step helps the model learn curves, textures, and layered patterns. Common activation functions are ReLU Sigmoid Tanh

Pooling Layer

The pooling layer reduces the size of the feature maps created by the previous convolutional neural network layers. It selects the most important values from small regions and leaves out the rest. This keeps essential details while removing noise and extra information. The result is a lighter and faster model that still retains the core features needed for learning. Pooling also helps the CNN stay stable when objects shift slightly within an image. A feature that appears a little to the left or right will still be captured after pooling. This makes the model more reliable and improves its ability to generalize.

How It Works

The layer divides each feature map into small blocks. Depending on the pooling type, it either picks the strongest value or takes the average of the block. This reduces the spatial size but keeps the important signal intact. Two common types

- Max pooling
- Average pooling

What It Keeps

Max pooling Strongest value in the region
Average pooling Average value in the region

Fully Connected Layer

After the earlier CNN layers extract and refine features, the CNN needs a way to combine everything into a final understanding. This is where the fully connected layer comes in. The model first flattens all feature maps into one long vector. This vector represents every detail learned so far. The flattened vector is then passed into dense units. Each unit connects to every value in the vector. These connections help the model understand how different features relate to each other. Early layers of CNN focus on edges and shapes. This layer focuses on the big picture, such as identifying whether the image shows a digit, an object, or a face.

How It Works

Each dense unit receives all input values and multiplies them with learned weights. The results are added and then passed through an activation function. This helps the model capture high-level patterns that simpler layers cannot detect. What this layer handles

Combining all learned features
Understanding high-level patterns
Passing values to the final stage

Output Layer

The output layer is the final step in the architecture of Convolutional Neural Network. This is where the model makes its decision. After the fully connected layer processes all features, the output layer converts those values into clear probabilities. These probabilities tell you which class the model believes the image belongs to.

For multi-class problems, the model uses Softmax. It assigns a probability to every class and ensures the values add up to one. For binary problems, the model uses Sigmoid. It produces a single probability that represents a yes or no outcome.

How It Works

The final values from the dense units are passed into the chosen output function. The function transforms raw numbers into interpretable scores. The class with the highest score becomes the predicted label. Role of this layer

Producing class scores
Returning the final prediction
Output Function

Use Case

Softmax Many classes
Sigmoid Yes or no task
These five layers in CNN complete the full flow of a simple CNN architecture. The model begins by detecting simple shapes and ends by delivering a clear prediction.

Filter-Kernel

1	0	9	3	0
3	6	0	0	5
5	3	8	2	0
1	5	7	9	7
4	8	2	2	0

5 × 5 image

CNN

Mathematical Form

$$8 \times 0 + 2 \times 1 + 0 \times 0 + 7 \times 3 + 9 \times 2 + 7 \times 1 + 2 \times 1 + 2 \times 3 + 0 \times 2$$

0	1	0
3	2	1
1	3	2

3 × 3 Kernel

51	58	22
65	74	77
55	64	56

3 × 3 Featured map

If **stride = 1**, the filter will move one pixel , see the below image:

Filter

If **stride = 2**, the filter will move two pixels, see the below image:

Filter

Processing of an Image in CNN

1

Feature Extraction:

This is the first major stage of the CNN architecture.

- The convolution layer scans small regions
- Filters detect edges, corners, and textures
- The activation layer highlights strong responses
- Pooling reduces the size and keeps the key signals

These steps create feature maps. Each map focuses on something the model finds useful.

3

Flattening and Preparation

After enough patterns are collected, the CNN flattens all feature maps into a single vector.

This vector becomes the input for the dense units, where the focus shifts from “what patterns exist” to “what those patterns mean.”

2

Building Higher-Level Patterns

Once simple features are learned, deeper layers in CNN combine them into richer patterns.

- Early layers catch thin edges
- Middle layers group edges into shapes
- Later layers form object parts or textures

Stacking these layers helps the model understand both local and global details.

4

Decision Stage

The dense layers of CNN learn the final relationships between features.

The output layer then converts the final values into probabilities.

Stage	Purpose
Feature extraction	Pick up edges and textures
Pattern building	Form shapes and object parts
Flattening	Prepare features for learning
Output	Produce class probabilities

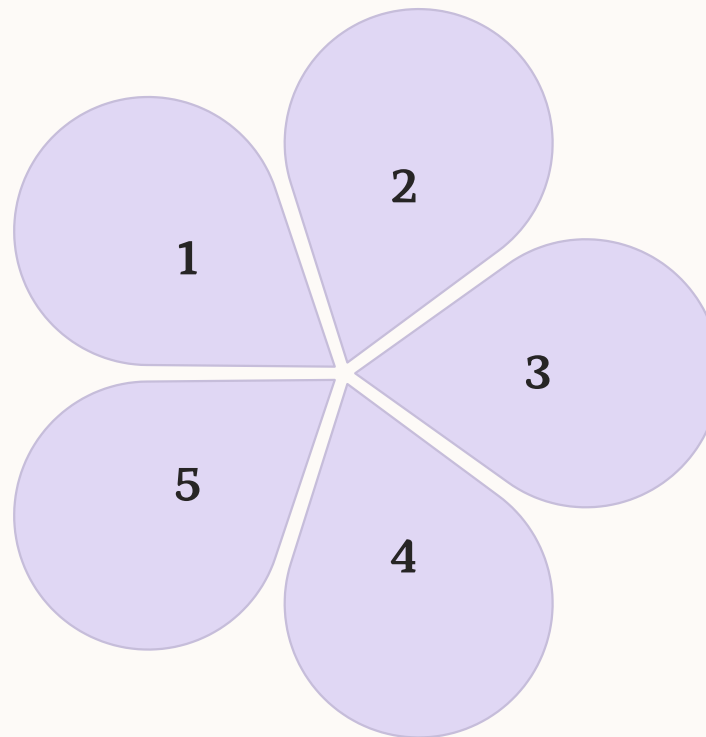
Performance Factors Influencing CNN Architecture

Number of Filters:

More filters capture more features.
Early CNN layers may use fewer filters,
while deeper layers use more to learn
detailed patterns.
Too many filters can slow training, so
balance matters.

Hardware and Training Limits:

Training deeper CNNs requires more
memory and processing power.
Batch size, learning rate, and training time
all depend on the available hardware.



Kernel Size:

Kernel size controls how much of the
image the model sees at once.
Small kernels capture fine details.
Larger kernels capture broader shapes but
increase computation.

Model Depth:

Adding more CNN layers lets the
model learn richer patterns.
Deep models work well for complex
tasks, but they need careful training to
avoid overfitting.

Regularization:

Regularization keeps the network stable
and prevents memorizing noise.

Common methods:

- Dropout
- Early stopping
- Weight decay

Variations and Extensions of Basic CNN Architecture

Deep CNNs:

Deep CNNs add more convolution and pooling layers.

More convolutional neural network layers let the model learn detailed and complex patterns.

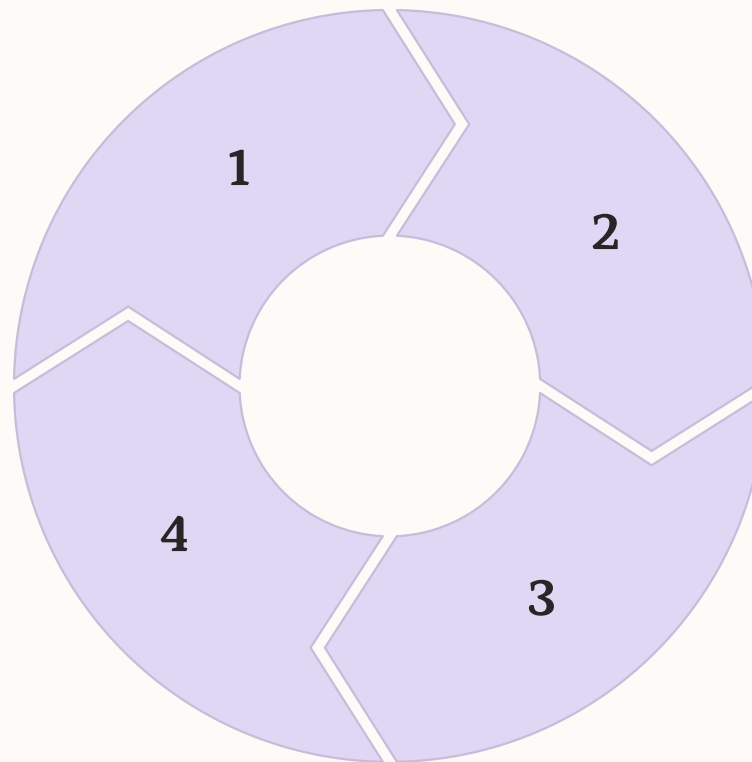
Early layers capture edges, while deeper layers understand shapes and object parts.

This design is common in large image classification tasks.

Depth-wise Separable Convolutions:

This variation breaks the convolution process into two smaller steps. It reduces computation and keeps the model fast.

Lightweight CNNs and mobile models often use this approach.



Residual Networks:

Residual networks help when models grow very deep.

They include skip connections that pass information forward without losing it.

This makes training stable and prevents the model from forgetting earlier patterns.

Dilated Convolutions:

Dilated convolutions widen the filter's field of view.

They help the model capture broader context without increasing parameters.

This works well for tasks like segmentation and depth estimation.

Residual learning using ResNet and its variants

Residual Networks, widely known as ResNets, are a specific type of [artificial neural network \(ANN\)](#) architecture designed to enable the training of extremely deep networks. Introduced by researchers at Microsoft in 2015, ResNet solved a critical bottleneck in deep learning known as the [vanishing gradient problem](#).

In traditional networks, stacking more layers often led to performance saturation or degradation because the signal required to update [model weights](#) would fade away as it propagated backward through the layers. ResNet introduced "skip connections" (or residual connections), which allow data to bypass one or more layers and flow directly to subsequent processing stages. This innovation proved that deeper networks could be trained effectively, leading to significant breakthroughs in [computer vision \(CV\)](#) and becoming a foundational concept for modern architectures.

The Core Concept: Residual Learning

The defining feature of a ResNet is the "residual block." In a standard [convolutional neural network \(CNN\)](#), each layer attempts to learn a direct mapping from input to output. As networks grow deeper, learning this direct mapping becomes increasingly difficult.

ResNet changes this approach by formulating the learning objective differently. Instead of hoping each stack of layers learns the entire underlying mapping, the residual block forces the layers to learn the "residual"—or the difference—between the input and the desired output. The original input is then added back to the learned residual through a skip connection. This structural change implies that if an identity mapping (passing the input unchanged) is optimal, the network can easily learn to push the residuals to zero. This makes [deep learning \(DL\)](#) models much easier to optimize, allowing them to scale from dozens to hundreds or even thousands of layers.

Key Architecture Variants

Since its inception, several variations of ResNet have become standard benchmarks in the AI community.

- **ResNet-50:** A 50-layer version that utilizes a "bottleneck" design. This design uses 1x1 convolutions to reduce and then restore dimensions, making the network computationally efficient while maintaining high [accuracy](#).
- **ResNet-101 and ResNet-152:** Deeper variants with 101 and 152 layers, respectively. These are often used when computational resources allow for higher complexity to capture more intricate [feature maps](#).
- **ResNeXt:** An evolution of ResNet that introduces a "cardinality" dimension, splitting the residual block into multiple parallel paths, which improves efficiency and performance.

ResNet vs. Other Architectures

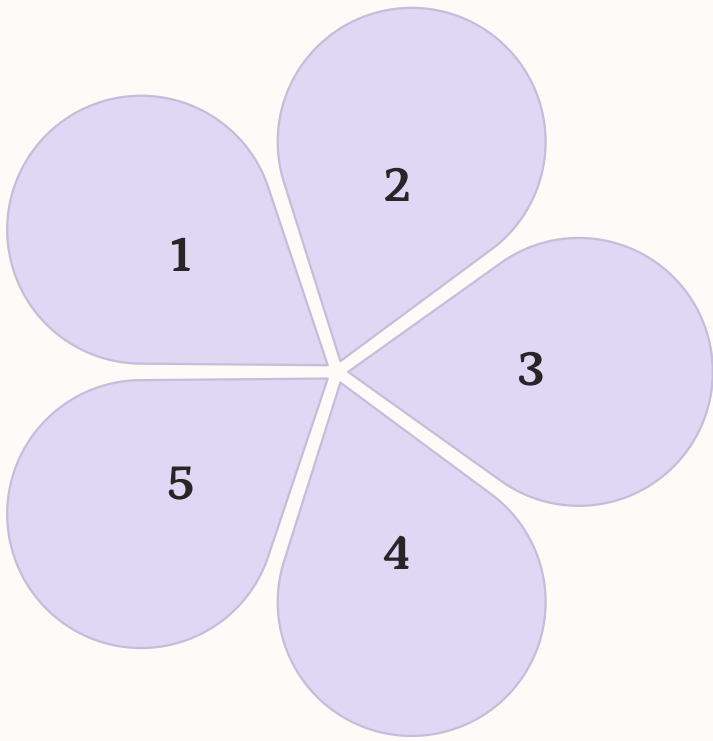
It is helpful to distinguish ResNet from other popular architectures to understand its specific utility.

- **ResNet vs. VGG:** VGG (Visual Geometry Group) networks are also deep CNNs but lack residual connections. Consequently, they are much harder to train at depths comparable to ResNet and are generally more computationally expensive due to their large fully connected layers.
- **ResNet vs. Inception:** Inception networks focus on width, using filters of multiple sizes within the same layer to capture features at different scales. ResNet focuses on depth. Modern architectures like Inception-ResNet combine both concepts.
- **ResNet vs. Vision Transformer (ViT):** While ViTs use self-attention mechanisms to process images globally, ResNets rely on local convolutions. However, ResNets remain a strong baseline and are often faster for smaller datasets or real-time [inference](#).

Architecture of ResNet:

Residual Block: A Residual Block contains:

- One or more convolutional layers.
- A skip connection that bypasses these layers
- Addition of input to convolution output
- This ensures uninterrupted flow of information and gradients



Global Average Pooling (GAP):

Before the final fully connected layer
ResNet uses GAP:

- Converts each feature map to a single value by averaging
- Reduces parameters less overfitting
- Produces compact feature representation

Skip (Shortcut) Connection

- Bypasses one or more layers .
- Adds input directly to output
- Prevents vanishing gradients
- Improves parameter updates

Handling Dimension Mismatch

When input and output dimensions differ:

- **Zero Padding:** Adds extra zeros to the input to match output dimensions in a residual block
- **Linear Projection:** Uses a learnable 1x1 convolution to match input and output dimensions for the skip connection.

Stacking Residual Blocks:

Multiple residual blocks can be stacked to create deep architectures. This allows networks to go very deep without suffering from degradation.

This problem of training very deep networks has been alleviated with the introduction of ResNet or residual networks and these Resnets are made up from Residual Blocks.

The very first thing we notice to be different is that there is a direct connection which skips some layers(may vary in different models) in between. This connection is called 'skip connection' and is the core of residual blocks. Due to this skip connection, the output of the layer is not the same now. Without using this skip connection, the input 'x' gets multiplied by the weights of the layer followed by adding a bias term.

Next, this term goes through the activation function, f() and we get our output as H(x).

$$H(x)=f(wx + b)$$

or $H(x)=f(x)$

Now with the introduction of skip connection, the output is changed to

$$H(x)=f(x)+x$$

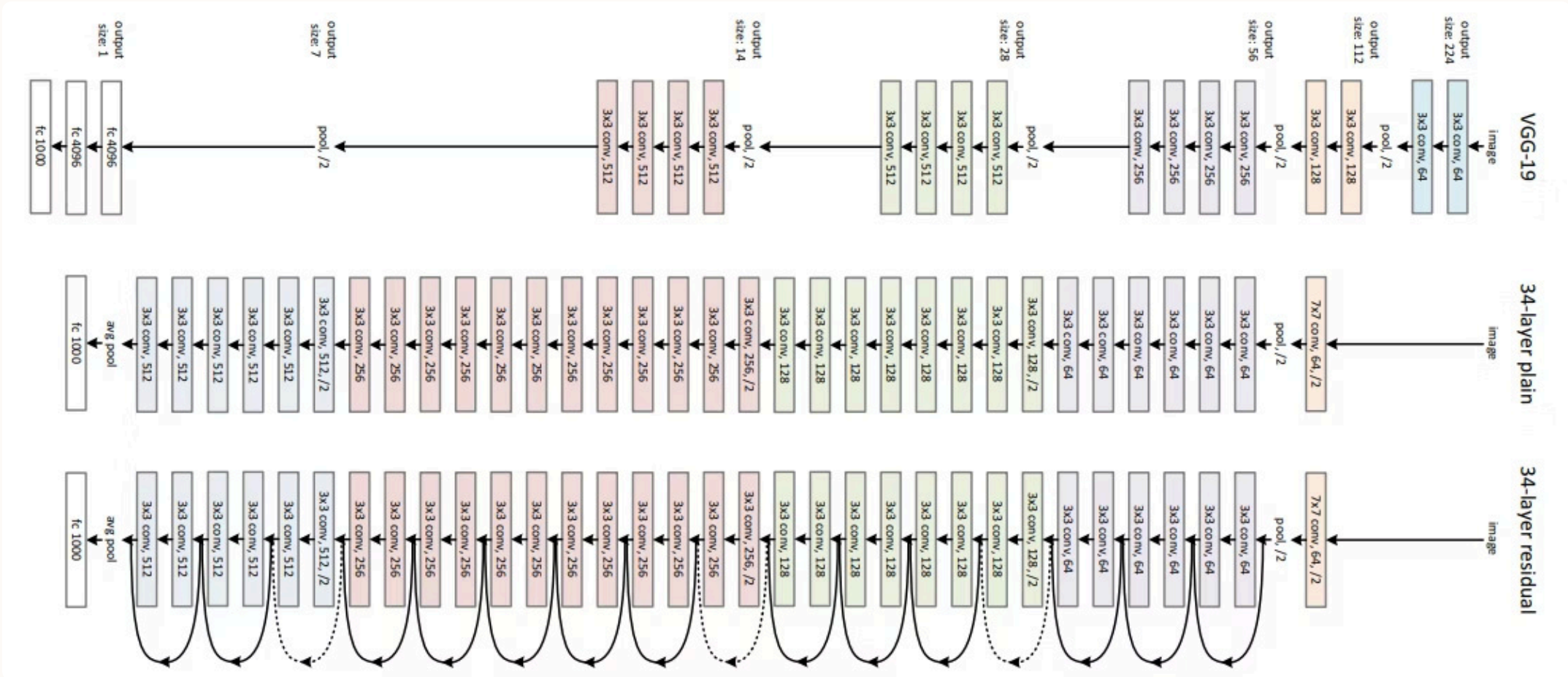
There appears to be a slight problem with this approach when the dimensions of the input vary from that of the output which can happen with convolutional and pooling layers. In this case, when dimensions of f(x) are different from x, we can take two approaches:

- The skip connection is padded with extra zero entries to increase its dimensions.
- The projection method is used to match the dimension which is done by adding 1x1 convolutional layers to input. In such a case, the output is:

$$H(x)=f(x)+w1.x$$

Here we add an additional parameter w1 whereas no additional parameter is added when using the first approach.

ResNet-34 is a deep residual network built on a 34-layer plain network inspired by VGG-19, with shortcut connections forming 16 residual blocks.



ResNet34

Here are the different stages of the ResNet-34 architecture, showing its structured arrangement of residual blocks.

- **First set:** 3 residual blocks each with 2 convolution layers of 64 filters and identity skip connections.
- **Second set:** 4 residual blocks each with 2 convolution layers of 128 filters uses zero-padding or 1x1 projections for dimension changes.
- **Third set:** 6 residual blocks, each with 2 convolution layers of 256 filters.
- **Fourth set:** 3 residual blocks with 2 convolution layers of 512 filters each.
- **Feature map:** Passed through Global Average Pooling a dense layer with 1000 neurons and softmax for classification.

DenseNet

Dense net is densely connected-convolutional networks. It is very similar to a ResNet with some-fundamental differences. ResNet is using an additive method that means they take a previous output as an input for a future layer, & in DenseNet takes all previous output as an input for a future layer as shown in the above image.

Key Characteristics of DenseNet

- Alleviated Vanishing Gradient Problem:** Dense connections ensure that gradients can flow directly to earlier layers, mitigating the vanishing gradient issue common in deep networks.
- Improved Feature Propagation:** Each layer has direct access to the gradients from the loss function and the original input signal, promoting better feature propagation.
- Feature Reuse:** By concatenating features from all preceding layers, DenseNet encourages feature reuse, reducing redundancy and improving efficiency.
- Reduced Parameters:** Despite its dense connections, DenseNet is parameter-efficient. It eliminates the need to relearn redundant features, resulting in fewer parameters compared to traditional networks.

Dense Block:

Dense blocks are the building blocks of DenseNet architectures. Each dense block consists of multiple convolutional layers, typically followed by batch [normalization](#) and a non-linear activation function (e.g., ReLU). Importantly, each layer within a dense block receives feature maps from all preceding layers as inputs, facilitating feature reuse and propagation.

Within a dense block, each layer receives the concatenated output of all preceding layers as its input. If a dense block has m layers, and each layer produces k feature maps (where k is known as the growth rate), the l -th layer will have

$k \times (l + l_0)$ input feature maps (where l_0 is the number of input channels to the dense block).

Example of a Dense Block:

- Layer 1:** Receives input feature maps.
- Layer 2:** Receives input feature maps + output of Layer 1.
- Layer 3:** Receives input feature maps + output of Layer 1 + output of Layer 2

This pattern continues for all layers within the block, ensuring a highly interconnected architecture.

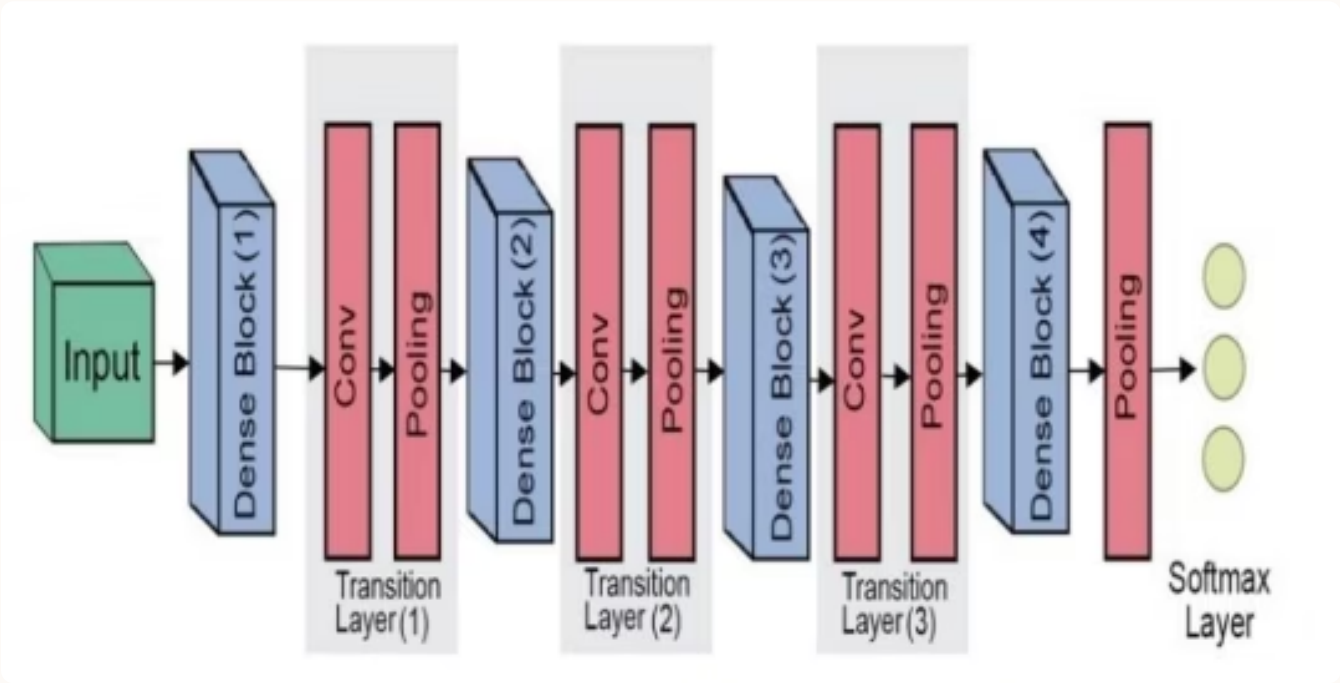
Transition Layer

Transition layers are used to connect dense blocks. They serve two main purposes: reducing the number of feature maps and downsampling the spatial dimensions of the feature maps. This helps in maintaining the computational efficiency and compactness of the network. A typical transition layer consists of:

- Batch Normalization:** Normalizes the feature maps.
- 1x1 Convolution:** Reduces the number of feature maps.
- Average Pooling:** Downsamples the spatial dimensions.

Growth Rate (k)

The growth rate (k) is a critical hyperparameter in DenseNet. It defines the number of feature maps each layer in a dense block produces. A larger growth rate means more information is added at each layer, but it also increases the computational cost. The choice of k affects the network's capacity and performance.



Inception Family

Inception is **a family of deep convolutional neural network (CNN) architectures developed by Google researchers**, starting with **GoogLeNet** (Inception v1) in 2014. The core innovation is the **Inception module**, which allows a network to perform multiple convolutional and pooling operations in parallel within the same layer rather than just stacking them sequentially.

Architecture:

Core Component: The Inception Module

The defining feature of this architecture is the **Inception module**, which performs multiple operations in parallel within the same layer rather than stacking them sequentially.

- **Parallel Processing:** Each module applies 1×1 , 3×3 and 5×5 convolutions alongside 3×3 max pooling simultaneously.
- **Multi-Scale Feature Extraction:** This design allows the network to capture features at various scales (fine to coarse) and "decide" which are most relevant.
- **Concatenation:** The outputs of these parallel paths are concatenated depth-wise to form the input for the next module.

Key Architectural Innovations

- **1×1 Convolutions (Dimensionality Reduction):** To manage the massive computational cost of 3×3 and 5×5 filters, 1×1 convolutions are used as "bottlenecks" to reduce the number of input channels before larger convolutions are applied.
- **Auxiliary Classifiers:** Because the network is 22 layers deep, it faces the **vanishing gradient problem**. To combat this, the authors added intermediate "auxiliary" branches that compute their own loss and push gradients back into earlier layers during training.
- **Global Average Pooling:** Instead of using multiple dense (fully connected) layers at the end—which are parameter-heavy—Inception uses global average pooling to transition to the final classifier, significantly reducing the model size.

Depthwise separable convolution

Depthwise separable convolution is an efficient convolutional neural network (CNN) layer that decomposes a standard convolution into two distinct steps: a [depthwise convolution](#) (spatial filtering) and a **pointwise convolution**.

Depthwise separable convolution is a technique used in efficient CNNs like **MobileNet** to **reduce computation and model size**

1. Depthwise Convolution

- Applies **one filter per input channel**
- No mixing of channel information
- Captures **spatial features** (edges, textures)

Input: $H \times W \times C$
Output: $H \times W \times C$

2. Pointwise Convolution (1×1 Conv)

- Uses **1×1 filters**
- Combines information across channels
- Learns **channel relationships**

Input: $H \times W \times C$
Output: $H \times W \times N$ (N = number of filters)

Combined Operation

👉 Depthwise + Pointwise = **Depthwise Separable Convolution**

Computational Advantage

Let:

- Input channels = C
- Output channels = N
- Kernel size = $K \times K$

Normal Convolution Cost:

 $K \times K \times C \times N$

Depthwise Separable Cost:

 $(K \times K \times C) + (C \times N)$

👉 Reduction: **~8–9× less computation**

Why it is important

- Faster training & inference
- Lower memory usage
- Suitable for mobile/edge devices
- Maintains good accuracy

Lightweight Model Design

Lightweight models are designed to work on:

- Mobile phones
- Embedded systems
- Real-time applications

Key Techniques

1. Depthwise Separable Convolutions

- Used in **MobileNet**

2. Width Multiplier (α)

- Reduces number of channels
- Smaller $\alpha \rightarrow$ lighter model

3. Resolution Multiplier

- Reduces input image size
- Less computation

4. Bottleneck Layers

- Reduce dimensions before heavy computation
- Used in **MobileNetV2**

5. Inverted Residuals

- Expand $\rightarrow$ process $\rightarrow$ compress
- Improves efficiency

6. Quantization

- Convert weights from float $\rightarrow$ int
- Reduces size and speeds up inference

7. Pruning

- Remove unnecessary weights/connections

Examples of Lightweight Models

- MobileNet
- SqueezeNet
- ShuffleNet
- EfficientNet (scaled versions).

MobileNet

MobileNet is a type of neural network architecture designed for mobile devices. It was developed by Google’s research team and first introduced in 2017. The primary goal of MobileNet is to provide high-performance, low-latency image classification and object detection on smartphones, tablets, and other resource-constrained devices.

MobileNet achieves this by using depthwise separable convolutions, which are a more efficient alternative to standard convolutions. Depthwise separable convolutions break down the computation into two separate steps: depthwise convolution followed by pointwise convolution.

By replacing regular convolutional layers with these depthwise separable convolutions and pointwise convolutions, MobileNet achieves high accuracy while minimizing computational overhead, making it well-suited for mobile devices and other resource-limited platforms.

Key Aspects of MobileNet:

- **Architecture:** Utilizes depthwise separable convolutions (factorizing standard convolutions into depthwise and pointwise convolutions).
- **Variants:**
 - **MobileNetV1:** Introduced depthwise separable convolutions.
 - **MobileNetV2:** Introduced inverted residual blocks and linear bottlenecks for better accuracy and efficiency.
 - **MobileNetV3:** Redesigned layers using hardware-aware network architecture search (NAS) and NetAdapt.
- **Hyperparameters:** Includes width multiplier (controls channels) and resolution multiplier (controls input size) to trade off latency vs. accuracy.
- **Applications:** Ideal for object detection, image classification, semantic segmentation, and facial recognition on mobile devices, drones, and IoT

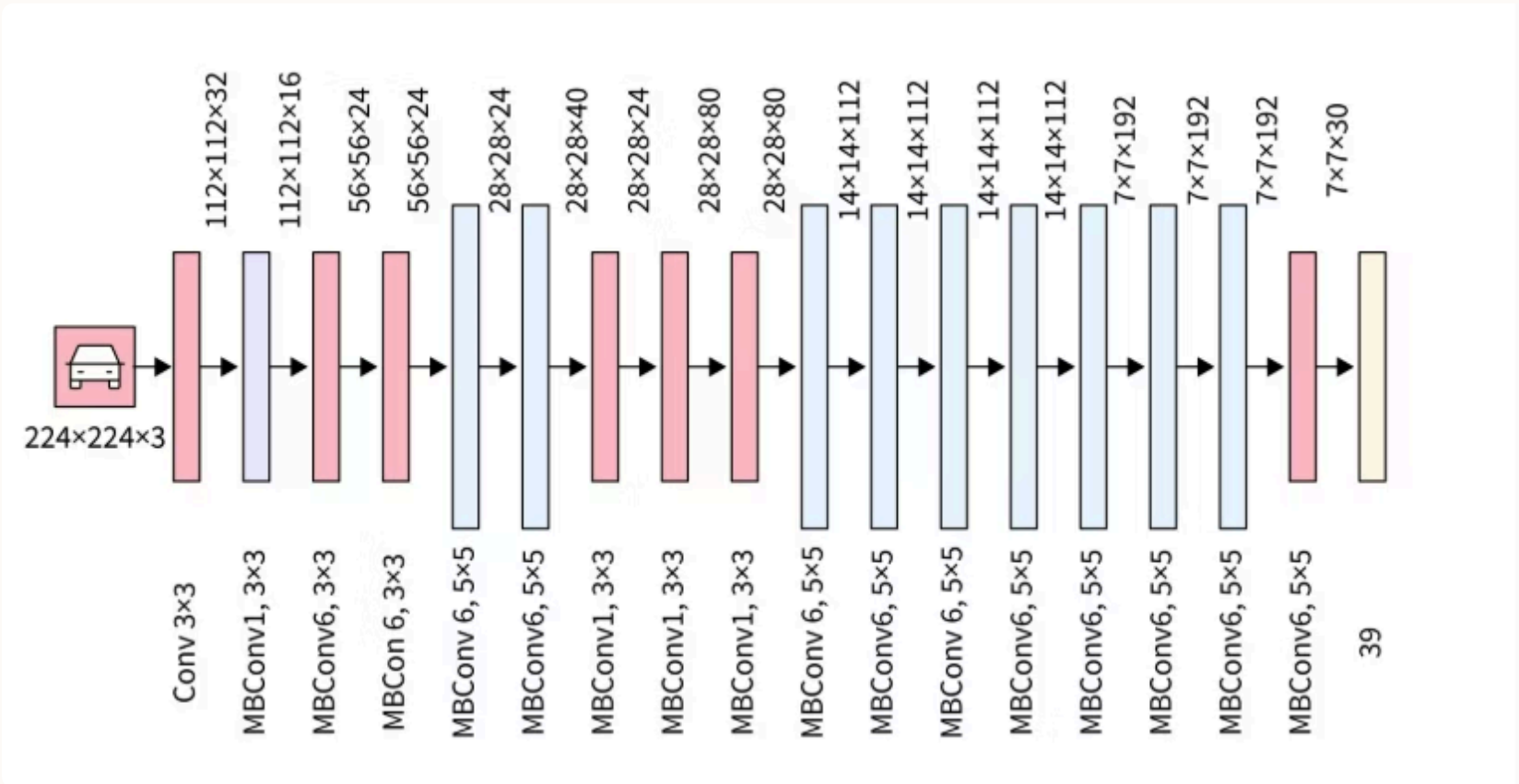
MobileNet Architecture



EfficientNet

EfficientNet is a family of [convolutional neural networks \(CNNs\)](#) that aims to achieve high performance with fewer computational resources compared to previous architectures.

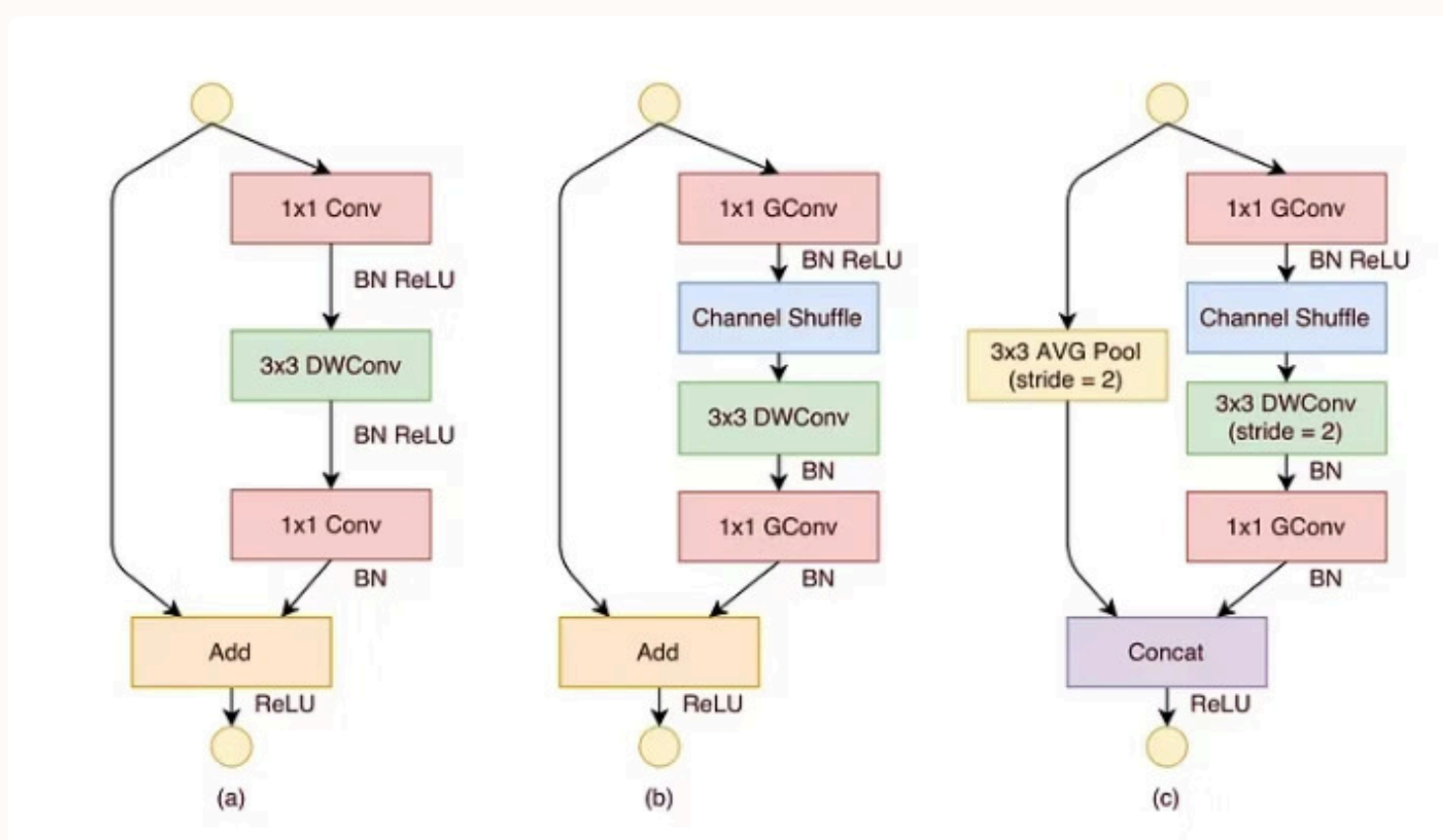
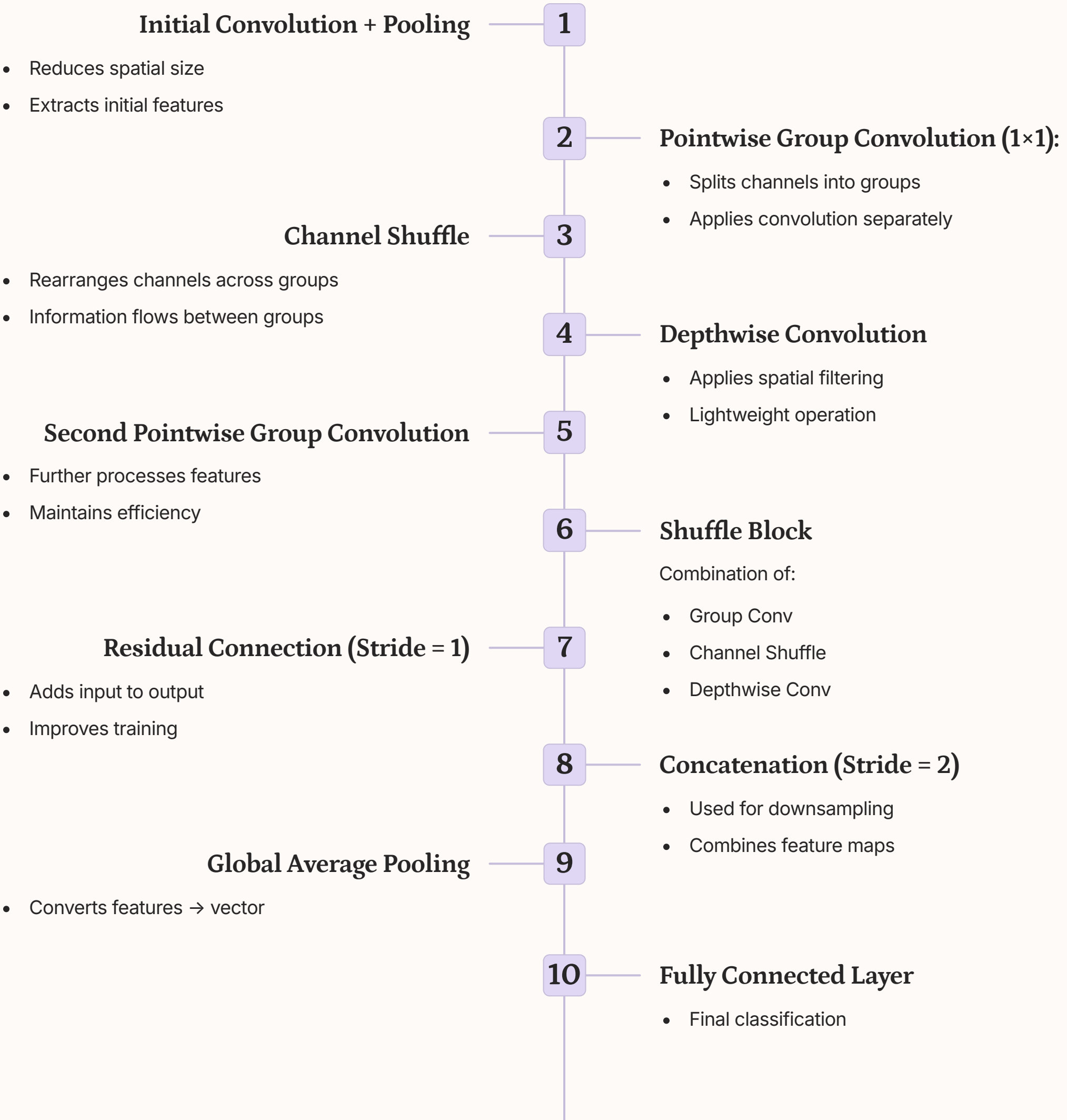
EfficientNet uses a technique called compound coefficient to scale up models in a simple but effective manner. Instead of randomly scaling up width, depth or resolution, compound scaling uniformly scales each dimension with a certain fixed set of scaling coefficients.



ShuffleNet

ShuffleNet is an extremely computation-efficient convolutional neural network (CNN) designed specifically for mobile devices with limited power, such as smartphones, robots, and drones.

It optimizes performance by using **pointwise group convolutions** and a novel **channel shuffle** operation to maintain high accuracy while drastically reducing computational costs.



Feature Pyramid Networks

A Feature Pyramid Network (FPN) is a specialized architectural component used in modern [computer vision \(CV\)](#) systems to improve the detection of objects at various scales. It effectively solves a longstanding challenge in image analysis: recognizing both large, prominent structures and tiny, distant details within the same image

By generating a multi-scale representation of the input—conceptually similar to a pyramid—FPNs allow neural networks to extract rich semantic information at every level of resolution.

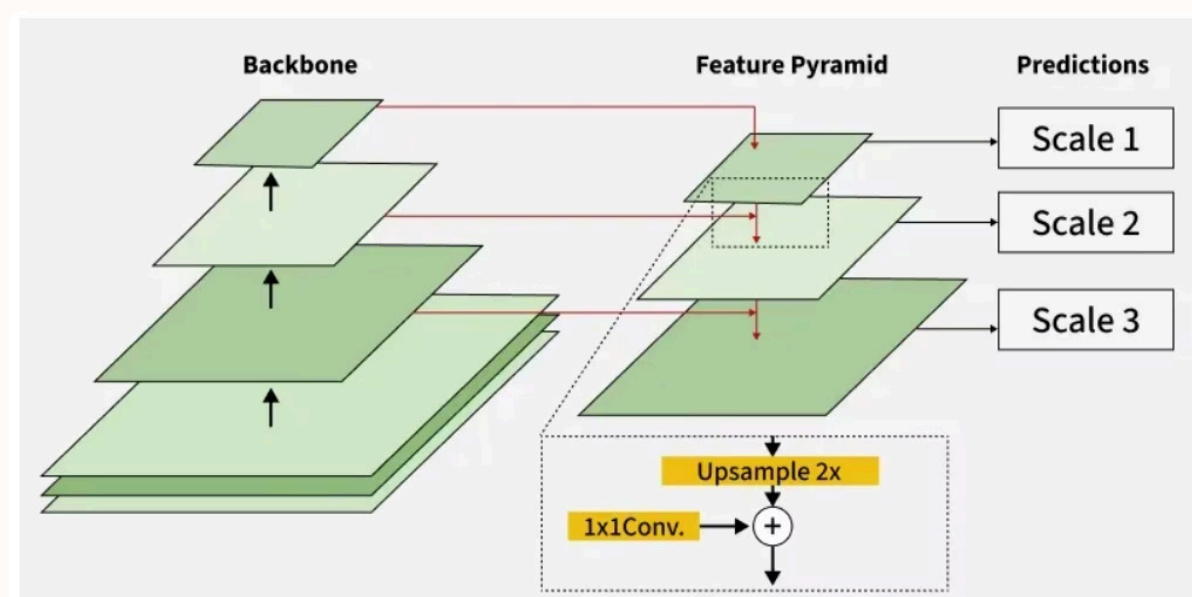
How Feature Pyramid Networks Work

The core innovation of the FPN lies in how it processes information. Traditional [Convolutional Neural Networks \(CNNs\)](#) naturally create a hierarchy of features where the input image is progressively downsampled. While this deepens the semantic understanding, it often degrades spatial resolution, making small objects vanish.

FPNs address this through a three-step process:

1. **Bottom-Up Pathway:** This is the standard feed-forward pass of the network, such as a [Residual Network \(ResNet\)](#). As the network processes the image, it creates [feature maps](#) that decrease in size but increase in semantic value.
2. **Top-Down Pathway:** The network constructs a higher-resolution pyramid by upsampling the semantically rich features from the deeper layers. This step "hallucinates" strong context back into larger spatial maps.
3. **Lateral Connections:** To recover the crisp details lost during downsampling, FPNs fuse the upsampled features with the original, high-resolution maps from the bottom-up pathway via lateral connections.

This combination results in a pyramid where every level has strong semantics *and* good localization, significantly boosting [precision](#) and [recall](#) across all object sizes.



Transfer Learning techniques

Transfer Learning :

Transfer learning is a [machine learning](#) technique in which knowledge gained through one task or dataset is used to improve model performance on another related task or different dataset.¹ In other words, transfer learning uses what has been learned in one setting to improve generalization in another setting.

What are the benefits of transfer learning?

TL offers several of the following benefits to researchers creating ML applications.

Enhanced efficiency

Training ML models takes time as they build knowledge and identify patterns. It also requires a large data set and is computationally expensive. In TL, a pre-trained model retains fundamental knowledge of tasks, features, weights, and functions, allowing it to adapt to new tasks faster. You can use a much smaller dataset and fewer resources while achieving better results.

Increased accessibility

Building deep-learning neural networks requires large data volumes, resources, computing power, and time. TL overcomes these barriers to creation, allowing organizations to adopt ML for custom use cases. You can adapt existing models to your requirements at a fraction of the cost. For example, using a pre-trained image recognition model, you can create models for medical imaging analysis, environmental monitoring, or facial recognition with minimal adjustments.

Improved performance

Models developed through TL often demonstrate greater robustness in diverse and challenging environments. They better handle real-world variability and noise, having been exposed to a wide range of scenarios in their initial training. They give better results and adapt to unpredictable conditions more flexibly.

Transfer learning strategies

Transductive transfer learning

Transductive transfer learning involves transferring knowledge from a specific source domain to a different but related target domain, with the primary focus being on the target domain. It is especially useful when there is little or no labeled data from the target domain.

Transductive transfer learning asks the model to make predictions on target data by using previously-gained knowledge. As the target data is mathematically similar to the source data, the model finds patterns and performs faster.

Inductive transfer learning

Inductive transfer learning is where the source and target domains are the same, but the tasks the model must complete differ. The pre-trained model is already familiar with the source data and trains faster for new functions.

Unsupervised transfer learning

Unsupervised transfer learning uses a strategy similar to inductive transfer learning to develop new abilities. However, you use this form of transfer learning when you only have unlabeled data in both the source and target domains.

The model learns the common features of unlabeled data to generalize more accurately when asked to perform a target task. This method is helpful if it is challenging or expensive to obtain labeled source data.

Object detection using YOLO

YOLO (You Only Look Once) is a popular, high-speed **real-time object detection** algorithm that **processes entire images in a single neural network pass**. This single-stage approach makes it significantly faster than traditional two-stage detection methods like Faster R-CNN, allowing it to be used in applications such as autonomous vehicles and live surveillance systems.

How YOLO Works

The core idea behind YOLO is to frame object detection as a single regression problem, going directly from image pixels to bounding box coordinates and class probabilities.

1. **Grid System:** The input image is divided into a uniform $S \times S$ grid. Each cell in the grid is assigned the responsibility of detecting an object if the center of that object's actual bounding box falls within the cell.
2. **Bounding Box Prediction:** Each grid cell predicts a fixed number of bounding boxes (e.g., B boxes), their confidence scores, and conditional class probabilities. The confidence score reflects the likelihood that a box contains an object and how accurate the box is.
3. **Single Convolutional Pass:** A single, deep convolutional neural network (CNN) is run on the entire image to extract features and produce the final predictions (class probabilities and bounding box coordinates) all at once.
4. **Non-Maximum Suppression (NMS):** After the initial predictions, NMS is applied as a post-processing step to eliminate redundant or overlapping bounding boxes for the same object, leaving only the most confident and best-positioned boxes.

Image segmentation using U-Net.

U-Net is a specialized convolutional neural network (CNN) architecture widely used for **image segmentation, a process that assigns a class label to every pixel in an image**. Its distinctive U-shape, combining an encoder and a decoder with skip connections, allows for precise localization of objects and boundaries.

Key Features and Architecture

The U-Net architecture is designed to capture both contextual information and fine-grained spatial details. It consists of three main parts:

- **Contracting Path (Encoder):** This is a typical CNN that involves repeated application of convolutional layers and max pooling operations to downsample the image. This process extracts higher-level, more abstract features while reducing the spatial dimensions.
- **Expanding Path (Decoder):** This path performs the opposite operation, using transposed convolutions (or upsampling) to gradually restore the spatial resolution of the feature maps to the original image size. It reconstructs the image based on the extracted features.
- **Skip Connections:** These crucial connections directly link feature maps from the encoder to the corresponding layers in the decoder at the same spatial resolution. This fusion of low-level, high-resolution features (from the encoder) with high-level, semantic features (from the decoder) is key to the U-Net's ability to produce highly accurate, pixel-wise segmentation masks and preserve fine details and boundaries.